

PPS - Scala Stream Fusion

Leonardo Naddei

14 settembre 2026

Indice

1	Metaprogrammazione e generazione statica del codice in Scala 3	3
1.1	Multi-stage programming in Scala 3	3
1.2	Metaprogrammazione generativa e analitica	4
1.3	Dalle espressioni all'AST tipizzato	4
1.4	Tipi staged e tipi riflessivi	4
1.5	Beta reduction delle funzioni	5
1.6	Simboli, variabili generate e hygiene	6
2	Requirement specification	7
2.0.1	Requisiti di business	7
2.0.2	Modello di dominio	7
2.0.3	Requisiti funzionali utente	7
2.0.4	Requisiti funzionali di sistema	7
2.0.5	Requisiti non funzionali	7
2.0.6	Requisiti di implementazione	7
3	Design architetturale	8
4	Design di dettaglio	9
4.1	Design dell'API pubblica	9
4.1.1	Operazioni terminali	9
4.1.2	Creazione dello stream: <code>from</code>	11
4.1.3	Tipologie di stream	11
4.1.4	Operazione terminale <code>collect</code>	12
4.1.5	Configurazione compile-time e runtime	13
4.1.6	Import semplificato dell'API	13
4.2	Implementazione	13
4.3	Parsing	14
4.4	Enrichment	14
4.5	Traduzione in OPIr	14
4.6	Lowering	14
4.7	Testing	14
4.8	Benchmark	14
5	Retrospettiva	15
5.1	Utilizzo di LLM	15

Abstract

Il presente lavoro descrive la realizzazione di una libreria per Scala 3 che consenta di definire pipeline di elaborazione secondo un paradigma dichiarativo, ottenendo al contempo prestazioni comparabili a quelle di codice imperativo specializzato manualmente.

Basandosi sul concetto di stream fusion, introdotto in Stream Fusion: From Lists to Streams to Nothing at All, e sfruttando le funzionalità di metaprogrammazione offerte dalle macro di Scala 3, è stata sviluppata una libreria capace di trasformare pipeline dichiarative in codice specializzato ed efficiente generato a compile-time. La libreria offre inoltre un'architettura estendibile per la definizione di nuovi operatori terminali, permettendo di integrare nuovi comportamenti nella fase di fusione senza rinunciare alle ottimizzazioni statiche.

Tale generazione statica permette di eliminare i principali overhead tipici delle astrazioni funzionali su JVM. L'analisi del bytecode prodotto conferma la completa fusione delle operazioni sequenziali in un unico ciclo imperativo e l'inlining diretto delle lambda nel callsite. Ciò azzerava l'uso di istruzioni `invokedynamic`, elimina il dispatch indiretto (evitando call site polimorfici o megamorfici) e rimuove le allocazioni di collezioni intermedie, iteratori e closure, azzerando al contempo il boxing dei tipi primitivi.

Per certi aspetti, il modello ottenuto richiama il paradigma di genericità dei template C++: il codice viene specializzato in fase di compilazione rispetto ai tipi e alla specifica composizione delle operazioni, quando tali informazioni sono disponibili staticamente. Le macro di Scala aggiungono tuttavia una componente esplicita di metaprogrammazione, permettendo non soltanto di istanziare codice generico, ma anche di analizzare, trasformare e ottimizzare la struttura della pipeline prima della sua emissione.

I benchmark condotti tramite JMH mostrano uno speedup significativo e un consumo di memoria drasticamente ridotto rispetto alle librerie standard. Il risultato è un framework che preserva l'espressività, la composizionalità e la type safety tipiche della programmazione funzionale.

Capitolo 1

Metaprogrammazione e generazione statica del codice in Scala 3

Il sistema di metaprogrammazione di Scala 3 segue un approccio di *multi-stage programming*: una parte del programma viene eseguita durante uno stage precedente e produce codice destinato a uno stage successivo. Questo modello permette di costruire macro che generano codice ben tipato e safe per costruzione [?, ?].

Nel progetto sviluppato, la metaprogrammazione viene utilizzata in entrambe le sue forme principali: analitica, per ispezionare e destrutturare la pipeline scritta dall'utente, e *generativa*, per produrre il codice che ne realizza l'esecuzione.

1.1 Multi-stage programming in Scala 3

Le due primitive fondamentali del sistema di metaprogrammazione staged di Scala 3 sono le *quote* e gli *splice*.

Una quote

```
'{ expression }
```

costruisce una rappresentazione tipizzata del codice contenuto al suo interno. Se **expression** produce un valore di tipo **T**, il risultato della quote ha tipo

`Expr[T]`

e rappresenta codice che produrrà un valore di tipo **T** nello stage successivo.

Uno splice:

```
${ expression }
```

opera invece nella direzione opposta e inserisce all'interno del codice quotato il frammento di programma rappresentato da un'espressione `Expr[T]`.

Il classico esempio di generazione staged è l'elevamento a potenza con esponente noto durante la compilazione:

```
def powCode(x: Expr[Int], n: Int)(using Quotes): Expr[Int] =  
  if n == 0 then  
    Expr(1)  
  else  
    '{ $x * ${ powCode(x, n - 1) } }
```

La ricorsione di `powCode` viene eseguita durante l'espansione della macro. Per un esponente pari a tre, il frammento generato è concettualmente equivalente a

```
x * x * x * 1
```

e non contiene quindi né il ciclo né la ricorsione impiegata dal metaprogramma per costruirlo.

1.2 Metaprogrammazione generativa e analitica

Una macro puramente generativa costruisce un nuovo frammento di programma a partire da informazioni note durante la compilazione. Nel progetto è tuttavia necessario anche compiere l'operazione inversa: ricevere un frammento di programma, riconoscerne la struttura e decomporlo nei suoi componenti.

Scala 3 supporta questa forma di metaprogrammazione attraverso il pattern matching sul codice quotato. Ad esempio, una generica espressione può essere analizzata con un pattern della forma

```
expr match {  
  case '{ someFunction($argument) } =>  
    ...  
}
```

Per i casi più semplici sarebbe possibile eseguire questa analisi utilizzando esclusivamente quoted pattern. Nel progetto, tuttavia, è necessario accedere a dettagli più fini dell'albero generato dal compilatore, rendendo necessario il passaggio alla Reflection API.

1.3 Dalle espressioni all'AST tipizzato

Il tipo `Expr[T]` offre un'interfaccia ad alto livello sul codice. Tale rappresentazione nasconde parte della struttura interna dell'espressione. Quando è necessario analizzare direttamente il programma è possibile convertire una `Expr[T]` nel relativo nodo dell'AST prodotto dal compilatore:

```
expr.asTerm
```

La Reflection API espone quindi la struttura tipizzata prodotta dal compilatore. Tra i nodi più importanti utilizzati nel parser vi sono:

- `Apply`, per l'applicazione di una funzione;
- `TypeApply`, per un'applicazione contenente parametri di tipo;
- `Select`, per la selezione di un membro;
- `Ident`, per il riferimento a un identificatore;
- `Block`, per un blocco contenente dichiarazioni e un'espressione finale;
- `Inlined`, per rappresentare il risultato di un'espansione inline;
- `ValDef`, `Assign` e `Ref`, utilizzati durante la generazione di nuovo codice.

Questo livello di rappresentazione è sostanzialmente più espressivo rispetto alle sole quote.

1.4 Tipi staged e tipi riflessivi

Scala 3 mette a disposizione due rappresentazioni complementari:

`Type[T]` rappresenta il tipo `T` nell'API staged ed è utilizzabile insieme a `Expr[T]`, quote e splice;

`TypeRepr` rappresenta invece un tipo all'interno della Reflection API e permette di ispezionarne direttamente la struttura.

Data una `TypeRepr`, il metodo `.asType` restituisce un valore `Type[?]` sul quale è possibile eseguire pattern matching:

```
item.asType match {
  case '[elem] => ...
}
```

Un esempio è l'estrazione del tipo per la creazione di un iteratore da un singolo elemento:

```
private def createSourceOf(item: Term): ParsedTree = {
  item.asType match {
    case '[elem] =>
      val singleExpr = item.asExprOf[elem]
      val iterableExpr: Expr[Iterable[elem]] =
        '{ Iterable.single[elem]($ { singleExpr }) }
      ...
  }
}
```

Questo esempio contiene in pochi passaggi l'intero collegamento tra reflection e staged programming:

1. viene determinato il tipo del termine dell'AST ricevuto;
2. il tipo viene estratto in `elem` mediante il pattern `'[elem]`;
3. il `Term` originale viene convertito in `Expr[elem]`;
4. viene generato un nuovo frammento `Expr[Iterable[elem]]`;

1.5 Beta reduction delle funzioni

Un altro elemento importante è l'utilizzo della β -reduction, considerando una funzione:

$$f = x \mapsto e, \quad (1.1)$$

l'applicazione

$$f(a) \quad (1.2)$$

può essere riscritta sostituendo direttamente x con a all'interno di e .

Scala 3 rende disponibile questa trasformazione attraverso: `Expr.betaReduce(...)`, viene utilizzata per l'applicazione esplicita delle lambda eliminando invocazioni tramite `invokedynamic`. Prendendo ad esempio l'operazione `filter`:

```
val cond = Expr.betaReduce('{ $ { filter.predicate }($elem) })
```

Se la funzione ricevuta è una lambda direttamente disponibile alla macro, il suo corpo può quindi essere incorporato nel codice generato.

Una pipeline come

```
stream
  .filter(x => x > 0)
  .map(x => x * 2)
```

può pertanto contribuire a generare un corpo concettualmente equivalente a

```
if (elem > 0) {
  emit(elem * 2)
}
```

Invece che:

```
if (filter.test(elem)) {  
    emit(map.apply(elem))  
}
```

Nell'ultimo caso, `filter` e `map` denotano istanze materializzate dalla JVM tramite call site `invokedynamic`, la loro applicazione avviene attraverso una chiamata indiretta all'interfaccia funzionale, ad esempio `Function1.apply`. Questo introduce un call site il cui target deve essere risolto e, qualora diventi polimorfo o megamorfo, può ridurre l'efficacia di devirtualizzazione, `inlining` e ulteriori ottimizzazioni interprocedurali del JIT. Incorporando staticamente il corpo della lambda nell'IR generato, la macro elimina sia la rappresentazione della funzione come entità invocabile sia il relativo dispatch.

1.6 Simboli, variabili generate e hygiene

Nell'AST di Scala, un riferimento a una variabile è associato semanticamente al relativo `Symbol`. Il nome è solamente una delle proprietà del simbolo e non ne rappresenta l'identità. La `Reflection API` permette di creare nuove variabili attraverso: `Symbol.newVal(...)`, e di fare successivamente riferimento allo stesso simbolo attraverso `Ref(symbol)`. La definizione viene creata nell'AST tramite `ValDef(symbol, Some(initializer))`

La distinzione tra nome e simbolo garantisce inoltre l'*hygiene* della macro: variabili generate dalla macro non possono andare in conflitto con variabili omonime definite nel codice utente.

TODO: esempio creazione e uso

Capitolo 2

Requirement specification

- 2.0.1 Requisiti di business
- 2.0.2 Modello di dominio
- 2.0.3 Requisiti funzionali utente
- 2.0.4 Requisiti funzionali di sistema
- 2.0.5 Requisiti non funzionali
- 2.0.6 Requisiti di implementazione

Capitolo 3

Design architetturale

L'applicativo è composto da due aree: l'API utilizzabile e un compilatore che si occupa del parsing, ottimizzazione e creazione di codice ad hoc a partire del codice delle macro

Capitolo 4

Design di dettaglio

4.1 Design dell'API pubblica

Particolare attenzione è stata posta nella progettazione dell'API pubblica della libreria. L'obiettivo è stato quello di fornire un'interfaccia che garantisca contemporaneamente:

- la possibilità di descrivere pipeline sufficientemente espressive
- l'estendibilità, in particolare per quanto riguarda le operazioni terminali
- il maggior numero possibile di garanzie a compile time
- la minimizzazione del codice necessario per verificare dinamicamente la correttezza delle combinazioni di operazioni richieste dall'utente.

Dal punto di vista concettuale, ogni stream può essere scomposto in tre componenti principali:

1. una sorgente, generalmente costituita da una collezione o da un array;
2. una sequenza di operazioni intermedie (`map`, `filter`, `flatMap`,...)
3. un'operazione terminale

La gerarchia di tipi esposta dall'API riflette direttamente questa struttura. L'utilizzo di `trait sealed`, tipi parametrici, tipi opachi e vincoli sui parametri di tipo permette di rappresentare a livello statico le proprietà della pipeline. In questo modo il compilatore Scala può guidare l'utente durante la costruzione dello stream e impedire le combinazioni non supportate.

Allo stesso tempo, l'API mantiene un certo grado di estendibilità, in particolare consentendo all'utente di definire nuove operazioni terminali attraverso l'interfaccia dei collector.

4.1.1 Operazioni terminali

Le operazioni terminali sono rappresentate attraverso l'astrazione `Collector`. La sua definizione generale è:

```
trait Collector[E, B, R, S <: TerminationPolicy] extends CollectorBase[E, B, R]
```

I quattro parametri di tipo rappresentano rispettivamente:

- E: il tipo degli elementi ricevuti dallo stream;
- B: il tipo dello stato intermedio, o buffer, utilizzato durante l'accumulazione;

- **R**: il tipo del risultato finale prodotto dal collector;
- **S**: la politica di terminazione del collector.

Il comportamento fondamentale di un collector è definito da **CollectorBase**:

```
trait CollectorBase[E, B, R] {
  def supplier(): B
  def accumulator(buf: B, elem: E): Boolean
  def finisher(buf: B): R
}
```

Il metodo **supplier** crea lo stato iniziale utilizzato durante la raccolta, mentre **accumulator** aggiunge un nuovo elemento, il valore booleano restituito segnala che l'elaborazione può essere interrotta (si pensi a una **findFirst**). Infine, **finisher** trasforma lo stato intermedio nel risultato finale.

La politica di terminazione è rappresentata mediante la seguente gerarchia:

```
sealed trait TerminationPolicy
sealed trait Exhaustive extends TerminationPolicy
sealed trait ShortCircuiting extends TerminationPolicy
```

Un collector **Exhaustive** richiede l'elaborazione dell'intero upstream, mentre un collector **ShortCircuiting** può terminare non appena viene soddisfatta una determinata condizione.

Ad esempio, i collector **toList** e **findFirst** hanno rispettivamente i seguenti tipi:

```
def toList[T]: Collector[T, ListBuffer[T], List[T], Exhaustive]
def findFirst[T]: Collector[T, OptionBuffer[T], Option[T], ShortCircuiting]
```

Nel primo caso è necessario consumare tutti gli elementi dello stream prima di produrre la lista risultante. Nel secondo caso, invece, il primo elemento ricevuto è sufficiente a determinare il risultato e il collector può richiedere l'interruzione anticipata della pipeline. Questa proprietà viene codificata direttamente nel tipo del collector.

Collector combinabili e parallelismo

Per poter eseguire una riduzione in parallelo non è sufficiente che il collector sia esaustivo. Gli stati intermedi prodotti indipendentemente dalle diverse partizioni dello stream devono anche poter essere combinati, questa proprietà viene rappresentata dall'interfaccia:

```
trait Combinable[B] {
  def combine(left: B, right: B): B
}
```

Un collector utilizzabile come terminale di una pipeline parallela viene quindi definito come:

```
trait ParallelCollector[E, B, R]
  extends Collector[E, B, R, Exhaustive]
  with Combinable[B]
```

Il tipo **ParallelCollector** esprime pertanto staticamente due proprietà:

1. la necessità di elaborare completamente lo stream;
2. la possibilità di combinare i risultati intermedi ottenuti da più worker.

Il type system permette così di distinguere un collector genericamente valido in una pipeline sequenziale da un collector che possiede anche le proprietà necessarie per essere utilizzato in una pipeline parallela.

Collector specializzati

Alcune operazioni terminali particolarmente comuni dispongono di un trattamento specializzato da parte del compilatore di FusedStream. Due esempi sono `toArray` e `summing`:

```
opaque type ToArrayCollector[T] <: ParallelCollector
opaque type SummingCollector[T <: Summable] <: ParallelCollector
```

I relativi metodi sono inoltre annotati con `@compileTimeOnly`:

Questi collector non vengono utilizzati come normali implementazioni runtime di `Collector`. La loro presenza nella pipeline viene riconosciuta durante l'espansione della macro e utilizzata per selezionare una strategia di code generation dedicata.

Il tipo opaco permette di esporre verso l'esterno soltanto le proprietà rilevanti del collector. Ad esempio, entrambi risultano essere sottotipi di `ParallelCollector`, rendendoli compatibili con le pipeline parallele, senza esporre all'utente i dettagli della strategia utilizzata internamente dal compilatore.

La combinazione di tipi opachi e `@compileTimeOnly` permette quindi di utilizzare il tipo del collector come protocollo di comunicazione tra l'API pubblica e la macro di compilazione.

Nel caso di `toArray`, il compilatore può generare direttamente una strategia di accumulazione specializzata evitando, quando possibile, boxing e chiamate indirette attraverso l'interfaccia `Collector`. Analogamente, `summing` può essere trasformato in una riduzione diretta.

In questo modo l'API mantiene una sintassi uniforme:

```
stream.collect(Collector.toArray)
stream.collect(Collector.summing)
```

pur consentendo al compilatore di generare implementazioni sostanzialmente differenti e specializzate per le singole operazioni terminali.

4.1.2 Creazione dello stream: `from`

La costruzione di una pipeline avviene a partire dal metodo `from`. Oltre a identificare la sorgente degli elementi, il tipo restituito da `from` determina quali modalità di esecuzione possono essere richieste successivamente.

Non tutte le sorgenti possiedono infatti le caratteristiche necessarie per essere sfruttate parallelamente. L'API rappresenta questa proprietà attraverso il tipo restituito: una sorgente per la quale è disponibile l'esecuzione parallela viene rappresentata come un `ParallelizableSource[A]`:

```
sealed trait ParallelizableSource[A]
  extends SequentialStream[A] {

  def parallel(): ParallelStream[A]
}
```

Il metodo `parallel` non appartiene quindi genericamente a ogni `Stream`, ma solamente alle sorgenti per le quali la libreria dichiara esplicitamente il supporto alla parallelizzazione. Una pipeline costruita a partire da una sorgente non parallelizzabile non può essere convertita in una `ParallelStream`: l'operazione semplicemente non è disponibile nel tipo statico restituito all'utente.

4.1.3 Tipologie di stream

La modalità di esecuzione della pipeline è rappresentata attraverso una gerarchia di tipi `sealed`:

```
sealed trait Stream[A]
sealed trait SequentialStream[A] extends Stream[A]
sealed trait ParallelizableSource[A] extends SequentialStream[A]
```

```
sealed trait ParallelStream[A] extends Stream[A]
```

`Stream[A]` rappresenta l'insieme minimo delle operazioni disponibili indipendentemente dalla modalità di esecuzione:

```
def filter(pred: A => Boolean): Stream[A]
def map[B](f: A => B): Stream[B]
def flatMap[B](f: A => SequentialStream[B]): Stream[B]
```

I sottotipi ridefiniscono (in maniera covariantemente) il tipo restituito dalle operazioni, ad esempio:

```
sealed trait ParallelStream[A] extends Stream[A] {
  override def filter(pred: A => Boolean): ParallelStream[A]
  override def map[B](f: A => B): ParallelStream[B]
  override def flatMap[B](f: A => SequentialStream[B]): ParallelStream[B]
}
```

Una chiamata a `map` su una `ParallelStream[A]` produce quindi una `ParallelStream[B]`, impedendo che l'informazione relativa alla modalità di esecuzione venga persa lungo la pipeline.

Analogamente, una pipeline sequenziale:

```
sealed trait SequentialStream[A] extends Stream[A] {
  override def filter(pred: A => Boolean): SequentialStream[A]
  override def map[B](f: A => B): SequentialStream[B]
  override def flatMap[B](f: A => SequentialStream[B]): SequentialStream[B]
  // Non supportati in stream paralleli
  def skip(n: Int): SequentialStream[A]
  def limit(n: Int): SequentialStream[A]
}
```

La posizione di `skip` e `limit` esclusivamente all'interno di `SequentialStream` garantisce che queste operazioni non risultino disponibili su una `ParallelStream`.

4.1.4 Operazione terminale `collect`

L'operazione `collect` costituisce il punto nel quale la descrizione della pipeline viene trasformata nel codice che ne realizza l'esecuzione.

La distinzione statica tra `SequentialStream` e `ParallelStream` permette di imporre requisiti differenti sul collector terminale.

Una pipeline sequenziale può utilizzare genericamente un `Collector`, mentre una pipeline parallela richiede un `ParallelCollector`, cioè un collector esaustivo e dotato di una funzione di combinazione dello stato intermedio. Le firme corrispondono quindi a:

```
// Sequential stream
collect(collector: Collector[E, B, R, S]): R

// Parallel stream
collect(collector: ParallelCollector[E, B, R]): R
```

Questa distinzione permette di rifiutare staticamente espressioni concettualmente non valide come l'utilizzo di un collector short-circuiting non combinabile come terminale di una pipeline parallela.

Ad esempio, un'operazione come:

```
stream
  . parallel()
  . collect(Collector.findFirst)
```

non soddisfa il vincolo richiesto da `ParallelCollector`, poiché `findFirst` è tipato come:

```
Collector [
  T,
  OptionBuffer [T] ,
  Option [T] ,
  ShortCircuiting << non corretto
]
```

Al contrario:

```
stream
  . parallel ()
  . collect ( Collector.toArray )
```

è accettabile perché `ToArrayCollector[T]` è dichiarato come sottotipo di `ParallelCollector`.

4.1.5 Configurazione compile-time e runtime

La libreria distingue tra due tipologie di configurazione: `CompileConfig`, relativa alle scelte che influenzano la generazione del codice, e `RuntimeConfig`, relativa ai parametri utilizzati durante l'esecuzione.

Entrambe le configurazioni sono gestite tramite il meccanismo dei `given` di Scala. La libreria fornisce un comportamento di default, permettendo quindi di utilizzare `FusedStream` senza alcuna configurazione esplicita nei casi più comuni.

Quando necessario, l'utente può comunque sovrascrivere i valori predefiniti fornendo una propria istanza `given` della configurazione desiderata nel contesto della pipeline.

In questo modo l'API mantiene una configurazione implicita e poco invasiva, lasciando comunque la possibilità di personalizzare sia il comportamento in fase di compilazione sia quello in fase di esecuzione.

4.1.6 Import semplificato dell'API

Gli elementi dell'API pubblica vengono riesposti direttamente attraverso l'oggetto `FusedStream`. Questo permette di importare l'intera API necessaria con una singola istruzione:

```
import FusedStream.*
```

ottenendo così accesso sia alle funzioni per la costruzione delle pipeline sia ai collector terminali pronti all'uso. L'obiettivo è mantenere l'interfaccia compatta e immediata, evitando che l'utente debba conoscere o importare esplicitamente i diversi oggetti interni che compongono l'API.

Sintesi delle Operazioni e Proprietà Algebriche

In Tabella 4.1 viene riportato un quadro riassuntivo delle firme formali e delle caratteristiche strutturali delle operazioni analizzate.

4.2 Implementazione

algoritmo implementato (push stream)
ottimizzazioni applicate ai parametri
enrichment: bubbling delle dichiarazioni
generazione di codice
implementazione del filter

Tabella 4.1: Confronto formale delle operazioni trasformative su flussi di dati.

Operazione	Firma di Dominio	Tipo di Trasformazione	Cardinalità ($1 : M$)	Gestione Stato
Map	$A \rightarrow B$	$\mathcal{S}(A) \rightarrow \mathcal{S}(B)$	$1 : 1$	Stateless
Filter	$A \rightarrow \mathbb{B}$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateless
Limit	$\mathbb{N}_0$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateful
Skip	$\mathbb{N}_0$	$\mathcal{S}(A) \rightarrow \mathcal{S}(A)$	$1 : \{0, 1\}$	Stateful
FlatMap	$A \rightarrow \mathcal{S}(B)$	$\mathcal{S}(A) \rightarrow \mathcal{S}(B)$	$1 : N \ (N \geq 0)$	Stateless / Contextual

implementazione del flatmapping

es semplice:

IncIter(from: Int): infinite lazy iterator, each next returns the next

```
FusedStream .from(List(1,2,3,4,5)) .skip(3) .flatMap(n -> Stream.of(IncIter(n*10)).limit(3))
.limit(10) .toList()
```

// Must be converted to

i = 0 j = 0

```
for i in l: if(i++ >= 3) «< must inject condition on j too k = 0 iter = IncIter( i* 10) for
ii in iter if(k++ >= 3) break ——— This get injectd into the flatMap ——— if(j ++ >= 10)
break all push(ii) ———
```

esempio estensione con collector utilizzando i trait

4.3 Parsing

4.4 Enrichment

4.5 Traduzione in OPIr

4.6 Lowering

4.7 Testing

4.8 Benchmark

Capitolo 5

Retrospettiva

5.1 Utilizzo di LLM